

Linear Regression - VisionGATE Questions

GATE Data Science and Artificial Intelligence (GATE DA)

Topics Index

Click on any sub-topic below to navigate directly to it:

1. [Linear Algebra Foundations](#)
2. [Model Formulation](#)
3. [Loss Functions](#)
4. [Closed Form Solution](#)
5. [Gradient Descent](#)
6. [Feature Scaling](#)

1 Linear Algebra Foundations

- Q1.** Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Which of the following matrix operations are validly defined? (Select all that apply)
- (A) AB
 - (B) $B^T A^T$
 - (C) $A^T B$
 - (D) ABB^T
- Q2.** Let $x, y \in \mathbb{R}^n$ be column vectors and $A \in \mathbb{R}^{n \times n}$ be a real matrix. Which of the following expressions is *always* identically equal to $x^T A y$?
- (A) $y^T A^T x$
 - (B) $y^T A x$
 - (C) $x^T A^T y$
 - (D) Both A and B
- Q3.** Let $x, a \in \mathbb{R}^n$ be column vectors and $W \in \mathbb{R}^{n \times n}$ be a symmetric matrix. Which of the following gradient identities with respect to x are correct? (Select all that apply)
- (A) $\nabla_x(a^T x) = a$
 - (B) $\nabla_x(x^T W x) = W x$
 - (C) $\nabla_x(x^T W x) = 2W x$
 - (D) $\nabla_x(x^T x) = 2x$
- Q4.** Let $X \in \mathbb{R}^{m \times n}$. Which of the following statements is FALSE?
- (A) $\text{rank}(X^T X) = \text{rank}(X)$
 - (B) The null space (kernel) of X is identical to the null space of $X^T X$.
 - (C) If X has linearly independent columns, then $X^T X$ is invertible.
 - (D) $\text{rank}(X^T X)$ is always equal to m .
- Q5.** Consider a square matrix $A \in \mathbb{R}^{n \times n}$. Which of the following conditions independently guarantee that A is invertible? (Select all that apply)
- (A) The determinant of A is non-zero ($\det(A) \neq 0$).
 - (B) The columns of A form a basis for $\mathbb{R}^n$.
 - (C) A has exactly n distinct real eigenvalues.
 - (D) The null space of A contains only the zero vector.

2 Model Formulation

- Q1.** In order to concisely represent a linear regression model with a bias term (i.e., $y = Xw + b$) as a single matrix-vector multiplication $y = X_{\text{aug}}w_{\text{aug}}$, what transformations must be applied to the design matrix $X \in \mathbb{R}^{N \times D}$ and the weight vector $w \in \mathbb{R}^D$? (Select all that apply)
- (A) A column of ones is appended to the design matrix X .
 - (B) A row of ones is appended to the design matrix X .
 - (C) The weight vector w is expanded by one dimension to include the bias scalar b .
 - (D) The target vector y is augmented with a column of ones.
- Q2.** In a multiple linear regression model formulated as $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon$, what is the precise interpretation of the estimated coefficient β_1 ?
- (A) The expected change in y for a one-unit increase in x_1 , ignoring the effect of x_2 .
 - (B) The correlation between the feature x_1 and the target y .
 - (C) The expected change in y for a one-unit increase in x_1 , assuming x_2 is held constant.
 - (D) The proportion of total variance in y that is exclusively explained by x_1 .
- Q3.** Which of the following statements regarding polynomial regression are true? (Select all that apply)
- (A) Polynomial regression is considered a linear model because the prediction is a linear combination of the model parameters (weights).
 - (B) Polynomial regression is considered a non-linear model because the loss function becomes non-convex with respect to the weights.
 - (C) By utilizing a polynomial feature mapping, the model can learn non-linear relationships or decision boundaries in the original feature space.
 - (D) Polynomial regression cannot be solved using the standard Ordinary Least Squares (OLS) closed-form solution.
- Q4.** Which of the following mathematical formulations can be solved using standard linear regression techniques by defining an appropriate feature transformation $\phi(x)$? (Select all that apply)
- (A) $y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$
 - (B) $y = w_0 + w_1 \sin(x) + w_2 \cos(x)$
 - (C) $y = w_0 + \exp(w_1 x)$
 - (D) $y = w_0 + w_1 (x_1 \times x_2)$

- Q5.** Suppose you are training an unregularized Ordinary Least Squares (OLS) regression model. If you scale a specific feature x_1 in your dataset by multiplying it by a constant $c > 0$ (so $x_1^{\text{new}} = c \cdot x_1$), how will the corresponding newly estimated coefficient $\hat{\beta}_1^{\text{new}}$ change compared to the original $\hat{\beta}_1$?
- (A) It will be multiplied by c .
 - (B) It will be divided by c .
 - (C) It will remain completely unchanged because OLS is scale-invariant.
 - (D) It will be divided by c^2 .
- Q6.** Suppose you are fitting a linear regression model but you use a standard, non-augmented design matrix $X \in \mathbb{R}^{N \times D}$ and solve for $w = (X^\top X)^{-1} X^\top y$. What geometric restriction are you forcing upon the resulting prediction hyperplane?
- (A) It must be orthogonal to the target vector y .
 - (B) It must be strictly horizontal (zero slope) across all feature dimensions.
 - (C) It must pass exactly through the origin $(0, 0, \dots, 0)$.
 - (D) It will perfectly interpolate all data points, regardless of N .
- Q7.** Let $X \in \mathbb{R}^{N \times D}$ be a design matrix without a bias column. You create an augmented design matrix X_{aug} by appending a column of ones. Which of the following statements are mathematically true? (Select all that apply)
- (A) X_{aug} has dimensions $N \times (D + 1)$.
 - (B) The maximum possible rank of X_{aug} is $D + 1$ (assuming $N \geq D + 1$).
 - (C) If the original matrix X already contained a feature column where every entry was the constant 5, then $X_{\text{aug}}^\top X_{\text{aug}}$ will be non-invertible.
 - (D) Augmenting the design matrix requires the target vector $y \in \mathbb{R}^N$ to also be augmented with an extra 1.
- Q8.** You fit a linear regression model to predict a house's price (y). One of the features is a binary variable x_{pool} , where $x_{\text{pool}} = 1$ if the house has a pool, and 0 otherwise. What is the interpretation of the estimated coefficient $\hat{\beta}_{\text{pool}}$?
- (A) The average price of all houses with a pool in the dataset.
 - (B) The expected difference in price between a house with a pool and a house without a pool, assuming all other features in the model are held constant.
 - (C) The probability that a house has a pool, given its price.
 - (D) The percentage increase in the house's price due to the presence of a pool.

- Q9.** In a multiple linear regression model, you discover that two features, x_1 and x_2 , are highly correlated (multicollinearity). How does this impact the interpretation of their respective coefficients, β_1 and β_2 ? (Select all that apply)
- (A) It becomes difficult to isolate the individual effect of x_1 because holding x_2 constant while changing x_1 is not reflective of the real data distribution.
 - (B) The true values of β_1 and β_2 become mathematically impossible to compute, as the OLS algorithm will fail to converge.
 - (C) The estimated coefficients $\hat{\beta}_1$ and $\hat{\beta}_2$ may become highly unstable and exhibit large standard errors.
 - (D) The coefficients will automatically shrink to zero to compensate for the correlation.

3 Loss Functions

- Q1.** In the comparison of the L_1 and L_2 losses, which property do the two losses *not* both satisfy?
- (A) Convexity
 - (B) Symmetry
 - (C) Differentiability
 - (D) Non-negativity
 - (E) Monotonicity
- Q2.** Why is convexity a desirable property of a loss function?
- (A) It guarantees that the loss is differentiable everywhere
 - (B) It guarantees that any local minimum is also the global minimum
 - (C) It guarantees that the loss is bounded above
 - (D) It guarantees that the solution is unique for any dataset
- Q3.** A fit produces residuals of magnitude 1, 1, 1, 1 and 10. What share of the total loss does the single large residual account for under L_1 and under L_2 respectively?
- (A) About 71% under L_1 , about 96% under L_2
 - (B) About 96% under L_1 , about 71% under L_2
 - (C) 20% under both, since it is one of five points
 - (D) About 50% under L_1 , about 90% under L_2
- Q4.** Given that L_1 is not differentiable at zero, why is it still used in practice?
- (A) It converges faster with gradient descent
 - (B) It has a simpler closed-form solution
 - (C) It is more robust to outliers, since it does not amplify large errors
 - (D) It is the only one of the two that is convex
- Q5.** What exactly goes wrong with the L_1 loss at zero error?
- (A) The derivative is zero, so the update stalls before reaching the minimum
 - (B) The derivative is undefined — it jumps from -1 to $+1$ with no value at the corner
 - (C) The derivative becomes infinite, so the step size explodes
 - (D) The derivative is well defined, but the loss stops being convex there
- Q6.** Why can we not simply use $\sum_i (y^{(i)} - \hat{y}^{(i)})$ as the loss?

- (A) The sum is not differentiable
 - (B) Positive and negative errors cancel, so a poor fit can score zero
 - (C) The sum is not convex in $\mathbf{w}$
 - (D) The sum grows with the number of data points
- Q7.** What does the monotonicity requirement on a loss function state?
- (A) The loss never decreases as training proceeds
 - (B) Over-prediction and under-prediction are penalised equally
 - (C) A prediction that is closer to the true value always incurs a smaller loss
 - (D) The loss increases without bound as the error grows
- Q8.** What is the effect of writing the objective as $J(\mathbf{w}) = \frac{1}{2n} \sum_i (y^{(i)} - \hat{y}^{(i)})^2$ rather than as the plain sum of squares?
- (A) It changes the location of the minimum, so $\mathbf{w}$ must be rescaled afterwards
 - (B) It makes the loss convex, which the raw sum is not
 - (C) It leaves the minimiser unchanged but alters the gradient magnitude, and so the useful range of α
 - (D) It has no effect at all on the optimisation
- Q9.** A loss that penalises predicting 10 when the truth is 8 exactly as much as predicting 6 when the truth is 8 is exhibiting which property?
- (A) Monotonicity
 - (B) Symmetry
 - (C) Convexity
 - (D) Zero at perfection

4 Closed Form Solution

- Q1.** To derive the normal equations for Ordinary Least Squares, we minimize the residual sum of squares $L(w) = (y - Xw)^\top (y - Xw)$. Which of the following represents the correct gradient $\nabla_w L(w)$ set to zero?
- (A) $-Xy^\top + X^\top Xw = 0$
 - (B) $-2X^\top y + 2X^\top Xw = 0$
 - (C) $-2Xy + 2XX^\top w = 0$
 - (D) $2X^\top y - X^\top Xw = 0$
- Q2.** Consider a dataset with a single feature and no intercept, where the feature matrix is $X = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and the target vector is $y = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. What is the exact closed-form OLS estimate for the weight w ?
- (A) $11/5$
 - (B) $5/11$
 - (C) $7/5$
 - (D) 2
- Q3.** Let $r = y - X\hat{w}$ be the residual vector from an OLS regression where $\hat{w}$ is the optimal closed-form solution. Which of the following statements are *always* true regardless of whether the model has an intercept? (Select all that apply)
- (A) $X^\top r = 0$
 - (B) $\hat{y}^\top r = 0$
 - (C) $\sum_{i=1}^n r_i = 0$
 - (D) $y^\top r = 0$
- Q4.** Let $H = X(X^\top X)^{-1}X^\top$ be the hat matrix (projection matrix) in OLS regression. Which of the following properties accurately describe H ? (Select all that apply)
- (A) H is an idempotent matrix (i.e., $H^2 = H$).
 - (B) H is a symmetric matrix (i.e., $H^\top = H$).
 - (C) $Hy = r$, where r is the residual vector.
 - (D) The diagonal elements h_{ii} represent the leverage of the i -th data point.
- Q5.** Let $X \in \mathbb{R}^{n \times d}$ be a feature matrix with n samples and d features (where $n \gg d$). What is the standard time complexity for computing the closed-form OLS solution $\hat{w} = (X^\top X)^{-1}X^\top y$?
- (A) $O(n^2d + d^3)$

- (B) $O(n^3 + d^3)$
- (C) $O(nd^2 + d^3)$
- (D) $O(nd + d^2)$

Q6. Consider the OLS closed-form solution $\hat{w} = (X^\top X)^{-1} X^\top y$. Which of the following conditions guarantee that a *unique* solution for $\hat{w}$ exists? (Select all that apply)

- (A) The feature matrix X has full column rank.
- (B) The number of samples n is strictly less than the number of features d .
- (C) $X^\top X$ is a positive definite matrix.
- (D) The columns of X are linearly independent.



VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION



VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

5 Gradient Descent

- Q1.** Consider the objective function $f(x) = x^2 - 4x$. You are performing standard Gradient Descent to find its minimum. If your starting point is $x_0 = 0$ and your learning rate is $\eta = 0.1$, what is the value of the parameter after one iteration (x_1)?
- (A) 0.4
 - (B) -0.4
 - (C) 0.8
 - (D) 4.0
- Q2.** Which of the following statements about the learning rate η in gradient descent are correct? (Select all that apply)
- (A) If η is set too large, the algorithm may overshoot the minimum and diverge, causing the loss to increase.
 - (B) If η is set very small, convergence is slow, but it guarantees finding the global minimum for any non-convex function.
 - (C) A learning rate schedule (e.g., step decay or exponential decay) can help the model converge more precisely by reducing η as it approaches the minimum.
 - (D) The optimal learning rate is a universal constant that must always fall strictly between 0 and 1.
- Q3.** Which of the following are valid stopping criteria for Gradient Descent, and correctly state their potential failure modes? (Select all that apply)
- (A) Stopping when the gradient norm $\|\nabla L(\theta)\| < \epsilon$ can prematurely halt training if the algorithm hits a saddle point in a non-convex loss landscape.
 - (B) Stopping when the training loss stops decreasing guarantees the model has optimal generalization to unseen data.
 - (C) Stopping when the parameter update magnitude $\|\theta_t - \theta_{t-1}\| < \epsilon$ can fail (stop too early) if the learning rate decays too aggressively.
 - (D) Early stopping based on validation loss can fail if the validation set is too small or unrepresentative of the true data distribution.
- Q4.** Suppose you are training a Linear Regression model on a dataset of 10,000 samples using Mini-Batch Gradient Descent. You choose a batch size of 100. How many total parameter updates will the model perform after exactly 5 epochs?
- (A) 5
 - (B) 100

- (C) 500
(D) 50,000
- Q5.** When comparing Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent (MBGD), which statements are true? (Select all that apply)
- (A) BGD computes the exact gradient of the empirical loss, leading to a stable trajectory, but it is computationally expensive per step on large datasets.
 - (B) SGD computes the gradient on a single data point, causing high variance in the trajectory which can occasionally help escape shallow local minima.
 - (C) MBGD generally exhibits higher variance in its gradient estimates than pure SGD.
 - (D) MBGD leverages modern hardware (like GPUs) better than pure SGD because it can compute gradients for multiple examples simultaneously via vectorized operations.
- Q6.** In the context of OLS Linear Regression, under what circumstances would you strongly prefer using Gradient Descent over the exact closed-form solution $\hat{w} = (X^\top X)^{-1} X^\top y$? (Select all that apply)
- (A) When the number of features d is extremely large (e.g., $d > 1,000,000$), making the inversion of $X^\top X$ computationally prohibitive.
 - (B) When you are operating in an "online learning" environment where new data arrives as a continuous stream over time.
 - (C) When you need the absolute exact mathematical minimum without any tolerance for approximation error.
 - (D) When the feature matrix X is perfectly orthogonal.
- Q7.** Fit $\hat{y} = wx$ to the points $(1, 3)$ and $(2, 5)$, starting at $w = 1$ with $\alpha = 0.1$. Using $\frac{\partial J}{\partial w} = -\frac{1}{n} \sum_i (y^{(i)} - \hat{y}^{(i)}) x^{(i)}$, what is w after one step?
- (A) $w = 0.6$
 - (B) $w = 1.4$
 - (C) $w = 0.2$
 - (D) $w = 1.8$
- Q8.** Run gradient descent on $J = 5w_1^2 + w_2^2$ from $(1, 1)$ with $\alpha = 0.1$. Where does one step land?
- (A) $(0, 0.8)$
 - (B) $(0.8, 0)$

- (C) (0.5, 0.9)
- (D) (0.9, 0.98)

Q9. What is a weakness of stopping when the gradient norm falls below a fixed threshold?

- (A) The gradient norm is never exactly zero, so it can never trigger
- (B) It stops too late, since the gradient vanishes only at the exact optimum
- (C) The threshold is not scale-free — rescaling the loss or the features changes what it means
- (D) It requires the loss to be non-convex to work

Q10. With $n = 1000$ and a mini-batch size of 50, how many weight updates occur over 10 epochs?

- (A) 10
- (B) 200
- (C) 10,000
- (D) 50

Q11. How does the per-epoch cost of stochastic gradient descent compare with that of batch gradient descent?

- (A) SGD is n times cheaper, since it uses one example per step
- (B) They cost the same, $O(nd)$ — the difference is one exact step versus n rough ones
- (C) Batch is cheaper, since it updates only once
- (D) SGD is more expensive by a factor of d

Q12. You have $n = 1,000,000$ rows and $d = 20$ features, all in memory. Which method is preferable?

- (A) The closed form, since d is small and the cost is dominated by a manageable nd^2 term
- (B) Gradient descent, since n is too large for the closed form
- (C) Gradient descent, since the d^3 term is prohibitive
- (D) Neither — the problem is too large to solve

Q13. Which of the following statements about stopping criteria are correct?
(Choose all the correct answers.)

- (A) A fixed iteration budget is simple but may stop short of convergence or waste steps after it
- (B) Small change in loss can fire on a flat plateau far from the optimum

- (C) A fixed gradient-norm threshold is not scale-free
- (D) Relative thresholds are more portable across problems than absolute ones
- (E) Monitoring validation loss instead can stop training before overfitting sets in
- (F) A small gradient norm guarantees you are near the global minimum for any loss function

Q14. With n examples and mini-batch size B , which of the following are correct? *(Choose all the correct answers.)*

- (A) An epoch is one full pass through the training data
- (B) An epoch contains $\lceil n/B \rceil$ updates
- (C) Batch gradient descent performs exactly one update per epoch
- (D) Pure SGD performs n updates per epoch
- (E) All three variants perform the same number of gradient *evaluations* per epoch
- (F) “Iteration” and “epoch” are interchangeable terms

Q15. Which of the following statements about batch, stochastic and mini-batch gradient descent are correct? *(Choose all the correct answers.)*

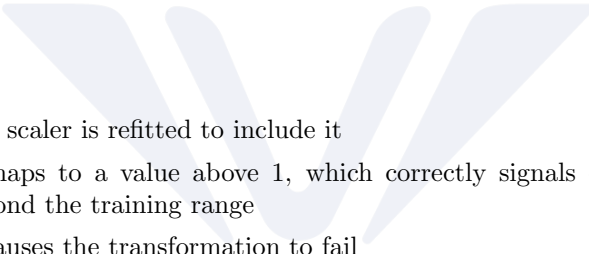
- (A) All three cost $O(nd)$ of gradient work per epoch
- (B) Batch gradient descent gives the exact gradient, so its loss curve decreases monotonically with a suitable α
- (C) SGD’s per-example gradients are unbiased estimates of the full gradient
- (D) Mini-batching reduces gradient noise by averaging, and suits vectorised hardware
- (E) SGD needs the whole dataset in memory at once
- (F) A decaying learning rate lets SGD settle rather than bounce indefinitely

6 Feature Scaling

- Q1.** A feature ranges from 10 to 50 in the training set. What is the min-max scaled value of $x = 30$?
- (A) 0.6
 - (B) 0.5
 - (C) 0.4
 - (D) 0.3
- Q2.** A training feature has mean 70 and standard deviation 10. What is the z-score of $x = 85$?
- (A) 0.15
 - (B) 8.5
 - (C) 1.5
 - (D) 15
- Q3.** What is the key difference between min-max scaling and standardization?
- (A) Min-max makes the distribution normal; standardization does not
 - (B) Min-max output is bounded to $[0, 1]$; standardized output is unbounded
 - (C) Standardization is a non-linear transformation; min-max is linear
 - (D) Standardization removes outliers; min-max preserves them
- Q4.** A feature has values mostly between 1 and 10, plus one recording error of 1000. What happens under min-max scaling?
- (A) Nothing — min-max scaling is robust to outliers
 - (B) The outlier maps to 0 and everything else spreads across $[0, 1]$
 - (C) The other values are all squashed into a narrow band near 0, losing resolution
 - (D) The transformation fails, since the range is undefined
- Q5.** One feature has values near 1 and another near 1000. What is the rough condition number of $X^T X$?
- (A) About 1000, matching the ratio of the feature scales
 - (B) About 10^6 , since the entries of $X^T X$ involve products of feature values
 - (C) About 1, since the condition number is unaffected by scaling
 - (D) About 2000, the sum of the two scales
- Q6.** Why does feature scaling speed up gradient descent?

- (A) It moves the minimum to a better location
- (B) It makes the loss convex, which it otherwise is not
- (C) It lowers the condition number, so one α suits all coordinates and the gradient points closer to the minimum
- (D) It reduces the per-iteration cost from $O(nd)$ to $O(n)$
- Q7.** A feature with standard deviation σ is standardized. Under plain OLS, what happens to its coefficient?
- (A) It is multiplied by σ
- (B) It is divided by σ
- (C) It is unchanged
- (D) It is multiplied by σ^2
- Q8.** After standardizing all features, what does the fitted intercept equal?
- (A) It becomes zero
- (B) It becomes the mean of y
- (C) It is unchanged
- (D) It becomes the sum of the feature means
- Q9.** Plain OLS is invariant to feature rescaling, yet ridge regression is not. Why?
- (A) Because $X^T X + \lambda I$ is otherwise singular
- (B) Because ridge has no closed-form solution on unscaled data
- (C) Because the penalty acts on the raw coefficient magnitudes, so features in large units get shrunk arbitrarily harder
- (D) Because ridge requires all features to lie in $[0, 1]$
- Q10.** You compute μ and σ over the combined training and test data before splitting. What is wrong with this?
- (A) Nothing — scaling is a deterministic transformation, so it cannot leak
- (B) Test-set information enters training, so the reported test error is optimistically biased
- (C) The training features will no longer have exactly zero mean
- (D) The coefficients become uninterpretable
- Q11.** You fit a min-max scaler on the training set. A test observation exceeds the training maximum. What happens?
- (A) It is clipped to 1.0

- (B) The scaler is refitted to include it
- (C) It maps to a value above 1, which correctly signals extrapolation beyond the training range
- (D) It causes the transformation to fail



VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION



VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION



VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

7 Feature Scaling

Key ideas for this section

[label=7.,leftmargin=2em]

1. **The two transformations.** Min-max maps to a fixed interval, standardization to zero mean and unit variance:

$$x' = \frac{x - \min}{\max - \min} \in [0, 1], \quad z = \frac{x - \mu}{\sigma}$$

Both are *affine*, so neither changes the shape of a distribution — skew and outliers survive intact.

2. **Why it helps optimisation.** The entries of $X^T X$ involve *products* of feature values, so a scale ratio of r between two columns becomes a curvature ratio of about r^2 . Scaling collapses that ratio, rounding the contours so a single α suits every coordinate.
 3. **Plain OLS is invariant.** Rescaling a feature by c divides its coefficient by c , leaving fitted values, residuals and R^2 untouched. The coefficients are precisely the part that absorbs the rescaling — which is what *produces* the invariance.
 4. **Regularization is not invariant.** Ridge and lasso penalise $\|\mathbf{w}\|$ directly, so an arbitrary choice of units decides how hard each coefficient is shrunk. Standardize before regularizing.
 5. **Fit the scaler on the training split only.** Statistics computed over training and test combined leak information and bias the reported error downward. Inside cross-validation, the scaler belongs inside each fold.
 6. **Two things not to scale.** The intercept column of ones, and any constant column (its σ is zero).
- Q1.** A feature ranges from 10 to 50 in the training set. What is the min-max scaled value of $x = 30$? B Apply the definition, subtracting the minimum and dividing by the *range*:

$$x' = \frac{x - \min}{\max - \min} = \frac{30 - 10}{50 - 10} = \frac{20}{40} = 0.5$$

The answer is unsurprising once noticed: 30 sits exactly halfway between 10 and 50, and an affine map preserves relative position, so it must land halfway between 0 and 1. This is worth remembering as a check — min-max scaling relocates and rescales the axis but never rearranges the points on it.

(A) 0.6 — **divides by the maximum instead of the range**, computing $30/50$ and forgetting to subtract the minimum. (C) 0.4 — **subtracts the minimum but still divides by the maximum**, computing $(30 - 10)/50$. Both errors come from confusing max with max – min; they agree only when min = 0. (D) 0.3 corresponds to no consistent calculation on these numbers. The denominator is the range, and it is determined by exactly two observations — the smallest and the largest. That is the structural reason min-max scaling is fragile: a single extreme value dictates the scale for the entire column, as the fourth question in this section shows. Standardization is steadier because σ averages over every observation, so one wild value moves it far less.

- Q2.** A training feature has mean 70 and standard deviation 10. What is the z-score of $x = 85$? C Subtract the mean, then divide by the standard deviation:

$$z = \frac{x - \mu}{\sigma} = \frac{85 - 70}{10} = \frac{15}{10} = 1.5$$

The value sits one and a half standard deviations above the mean. Note what the z-score buys: it is *dimensionless*. Whatever units x was measured in cancel between numerator and denominator, which is exactly why standardized coefficients become comparable across features while raw ones do not.

Applied to an entire column, the transformation gives that column mean 0 and standard deviation 1:

$$\mathbb{E}[z] = \frac{\mathbb{E}[x] - \mu}{\sigma} = 0, \quad \text{Var}[z] = \frac{\text{Var}[x]}{\sigma^2} = 1$$

(A) 0.15 — **divides by σ twice**, i.e. $15/100$. (B) 8.5 — **forgets to subtract the mean**, computing $85/10$. (D) 15 — **forgets to divide by σ** , leaving the raw deviation from the mean, which still carries the original units. Option (D) is the most instructive slip: 15 is a perfectly meaningful quantity — the deviation in the feature’s own units — but it is not standardized, because the division by σ is what removes the units. Centring alone gives mean zero; it is the scaling step that makes different features commensurable.

- Q3.** What is the key difference between min-max scaling and standardization? B Min-max output is confined to $[0, 1]$ by construction: the training minimum maps to 0, the training maximum to 1, and everything between lands in between. Standardization offers no such guarantee — a value five standard deviations from the mean maps to 5, and there is no upper limit at all. **Bounded versus unbounded** is the essential difference.

(A) — **false, and the most seductive misconception**. Neither method makes a distribution normal. Both are *affine* maps of the form

$x \mapsto ax + b$, which relocate and rescale the axis without rearranging any points on it. A strongly skewed column is exactly as skewed after scaling as before; what changes is the numbers on the axis, not the shape of the histogram. Making a distribution more symmetric requires a genuinely non-linear transformation such as a logarithm or a Box–Cox power transform.

(C) — **false, and reversed.** Both transformations are affine. Neither is non-linear, and the option assigns the property to the wrong one besides.

(D) — **false.** Neither method removes anything. An outlier remains an outlier under both; the two differ only in how much damage it does to the rest of the column, which is the subject of the next question. The bounded range in (B) is why min-max is preferred where a fixed input domain is required — image pixel intensities, or activations feeding a bounded non-linearity. Standardization is preferred almost everywhere else, both because it is less sensitive to extremes and because unit variance is the natural scale for the curvature argument that motivates scaling in the first place.

- Q4.** A feature has values mostly between 1 and 10, plus one recording error of 1000. What happens under min-max scaling? C The range is now dictated entirely by the erroneous value:

$$\max - \min = 1000 - 1 = 999$$

So the genuine data, spanning 1 to 10, maps to

$$\frac{1 - 1}{999} = 0 \quad \text{through} \quad \frac{10 - 1}{999} \approx 0.009$$

Every real observation is compressed into roughly the first 1% of the interval, where the differences between them become vanishingly small relative to the scale. The outlier alone occupies the remaining 99%. **The resolution among the values that matter has been destroyed** by a single bad point.

(A) — **false.** Min-max is the *least* robust common scaler, precisely because its denominator depends on only two observations. Standardization is steadier here — one extreme value among many raises σ but does not hijack it the way it hijacks a range — though it is not immune either. Genuinely robust alternatives use the median and interquartile range in place of the mean and standard deviation.

(B) — **false, and inverted.** Under min-max the maximum maps to 1, not 0. The outlier is the maximum, so it lands at 1 while everything else clusters near 0 — the opposite of the spread the option describes.

(D) — **false**. The range is well defined and non-zero, so the transformation completes without error. That is the difficulty: it fails silently, producing a numerically valid column that has lost the information it was supposed to carry. The practical lesson is ordering. Outlier detection belongs *before* scaling, not after, since scaling on contaminated data bakes the contamination into the transformation itself — and the stored min and max then get applied to every future observation. A related edge case: a genuinely constant column has $\max = \min$, giving division by zero. Such columns carry no information and are normally dropped.

- Q5.** One feature has values near 1 and another near 1000. What is the rough condition number of $X^\top X$? B The entries of $X^\top X$ are inner products of feature columns:

$$(X^\top X)_{jk} = \sum_{i=1}^n x_j^{(i)} x_k^{(i)}$$

Every entry is therefore a sum of *products* of feature values. The diagonal entry for the small feature is of order $n \cdot 1^2 = n$, while that for the large feature is of order $n \cdot 1000^2 = 10^6 n$. The curvature ratio is thus

$$\kappa \approx \frac{10^6 n}{n} = 10^6$$

The scale disparity is squared. A ratio of 1000 between the features becomes a ratio of 10^6 between the curvatures.

This is catastrophic for gradient descent. The contours are ellipses stretched by a factor of $\sqrt{10^6} = 1000$ along one axis, and the stability bound $\alpha < 1/\lambda_{\max}$ is set by the steep direction while progress in the shallow direction is governed by $\lambda_{\min}$. Any α safe for the first leaves the second essentially frozen.

(A) 1000 — **the feature ratio itself, not squared**. This is the most common answer and it misses the squaring, which is exactly the point of the question. (C) — **false**, and states the opposite of the truth: unequal scaling is the primary source of ill-conditioning. (D) 2000 — **adds the two scales**, but the condition number is a ratio of eigenvalues, not a sum of anything. This is why the unscaled example in the lecture notes detonates within five iterations rather than merely converging slowly. It also explains a numerical point that has nothing to do with gradient descent: forming $X^\top X$ squares the condition number of X , so even a direct solve loses precision on badly-scaled data. That is the argument for QR or SVD factorisations of X over explicitly building the Gram matrix.

- Q6.** Why does feature scaling speed up gradient descent? C Scaling equalises the diagonal entries of $X^\top X$, which pulls the elongated elliptical contours toward circles. Two things improve at once. First, the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ falls, so a **single α can serve every coordinate** instead

of being capped by the steepest while the shallowest crawls. Second, on rounder contours the negative gradient **points closer to the minimum**, rather than across a narrow valley, so the trajectory stops zigzagging.

(A) — false, and the key conceptual trap. Scaling does *not* move the minimum. Under plain OLS the fitted values, residuals and R^2 are identical before and after scaling; only the coefficients change, absorbing the rescaling so that the products $w_j x_j$ stay fixed. Scaling improves *how you get there*, never *where you end up*.

(B) — false. The squared-error loss is $\|\mathbf{y} - X\mathbf{w}\|^2$, a convex quadratic with curvature matrix $X^\top X$, which is positive semi-definite whatever the columns contain. Convexity holds regardless of scaling and is not something scaling supplies.

(D) — false. The per-iteration cost is $O(nd)$ either way, since the same matrix–vector products are computed on the same-sized arrays. What falls is the *number* of iterations required, not the cost of each. The distinction in (D) is worth stating precisely: scaling changes the iteration count, not the cost per iteration. And the distinction in (A) is the one this whole section rests on — for unregularized OLS, scaling is purely an optimisation convenience. The moment a penalty is added, it also changes the answer.

- Q7.** A feature with standard deviation σ is standardized. Under plain OLS, what happens to its coefficient? A Write the standardized feature as $x' = (x - \mu)/\sigma$, so that $x = \sigma x' + \mu$. Since OLS reaches the same fitted values either way, the contribution of this feature must be unchanged. Substituting,

$$wx = w(\sigma x' + \mu) = \underbrace{(w\sigma)}_{w'} x' + \underbrace{w\mu}_{\text{absorbed by the intercept}}$$

so the new coefficient is

$$w' = w\sigma$$

It is multiplied by σ . A feature with $\sigma = 20$ has its coefficient multiplied by 20.

The direction is worth pinning down by intuition rather than memory. Standardizing a widely-spread feature *shrinks* its numerical values, so its coefficient must *grow* to compensate and keep the product fixed. Dividing the feature by σ multiplies the weight by σ .

The interpretation shifts accordingly: w' is the change in $\hat{y}$ per *one-standard-deviation* rise in the feature. That is what makes standardized coefficients comparable to one another — they are all quoted per standard deviation rather than per unit of whatever the feature happens to be measured in.

(B) — **the right factor, wrong direction**, and the answer to the reverse question of what happens when a *feature* is scaled by c . (C) — **false**: OLS is invariant in its predictions, not in its coefficients. (D) — **false**: quadratic scaling applies to variances, $\text{Var}[aX] = a^2 \text{Var}[X]$, and to entries of $X^\top X$; a coefficient is a rate of change and scales linearly. To recover an original-unit coefficient from a standardized fit, invert the relation: $w = w'/\sigma$. This matters when reporting results, since a coefficient of “12,000 per standard deviation of floor area” is not a statement a client can act on. Note also that this is the same invariance as the earlier question on scaling a feature by c — standardization is just the case $c = 1/\sigma$ combined with a shift.

- Q8.** After standardizing all features, what does the fitted intercept equal? B Standardizing centres every feature column at zero. Now use a property of any least-squares fit with an intercept: because the residuals are orthogonal to the ones column, they sum to zero, and consequently the fitted surface passes through the point of means. Evaluating the model there,

$$\bar{y} = w_0 + \sum_j w_j \bar{x}'_j$$

and after centring every $\bar{x}'_j = 0$, so every term in the sum vanishes:

$$w_0 = \bar{y}$$

The intercept becomes the mean of the target.

This gives standardized coefficients a clean reading: the prediction for an observation at the average of every feature is simply the average target, and each w'_j says how far $\hat{y}$ moves from that average when feature j shifts by one standard deviation with all others held at their means.

(A) — **false, and confuses the features with the target**. Centring is applied to X , not to $\mathbf{y}$. The intercept would be zero only if the target were centred as well — which is in fact common practice, and then the intercept can be dropped from the model entirely. (C) — **false**: the intercept absorbs the shift, changing from w_0 to $w_0 + \sum_j w_j \mu_j$. (D) — **false**: the feature means enter the calculation, but the intercept is the target mean, not a sum of feature means. Two practical corollaries. First, the ones column must not itself be scaled — standardizing a constant column would divide by $\sigma = 0$, and centring it would annihilate the intercept altogether. Second, this is one of the two reasons the intercept is excluded from regularization penalties: it now represents the target’s overall level, which there is no reason to shrink toward zero.

- Q9.** Plain OLS is invariant to feature rescaling, yet ridge regression is not. Why? C The invariance argument for OLS works because the objective depends on $\mathbf{w}$ only through the product $X\mathbf{w}$. Replacing X with XM for

invertible M gives the solution $M^{-1}\mathbf{w}$, and $(XM)(M^{-1}\mathbf{w}) = X\mathbf{w}$, so the fit is untouched.

Ridge adds a term that breaks this:

$$J(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_2^2$$

The penalty is a function of $\mathbf{w}$ *directly*, not of $X\mathbf{w}$, so the substitution no longer leaves it invariant. Concretely: a feature measured in metres receives a coefficient 100 times larger than the same feature in centimetres, and since the penalty is quadratic it therefore absorbs 10^4 times more of the penalty budget. **An arbitrary choice of units ends up deciding which features get shrunk.** Standardizing first puts every coefficient on comparable footing so that λ applies evenly.

(A) — **false.** $X^\top X + \lambda I$ is invertible for every $\lambda > 0$ regardless of scaling, since $X^\top X$ is positive semi-definite and adding λI shifts all its eigenvalues up by λ . Guaranteed invertibility is a virtue of ridge, not a consequence of scaling.

(B) — **false.** Ridge has the closed form $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ on any data whatsoever, scaled or not.

(D) — **false.** Ridge imposes no constraint on the feature range. Standardization is advisable, but the $[0, 1]$ interval specifically is not required by anything in the method. This is the sharpest statement of when scaling is optional and when it is not. For plain OLS it changes the route but not the destination, so skipping it costs only iterations. For ridge and lasso it changes the destination — a different model is fitted — so skipping it is a modelling error. The same sensitivity is inherited by anything that behaves like an implicit regularizer, including early stopping: run to convergence, gradient descent reaches the invariant OLS minimum, but stopped at a fixed budget, where you land depends on the geometry that scaling alters.

Q10. You compute μ and σ over the combined training and test data before splitting. What is wrong with this? B The test set exists to estimate performance on data the model has never seen. Computing μ and σ over all rows means those statistics encode information about the test observations, and that information is then baked into every training feature the model fits on. **The reported test error is optimistically biased** — it no longer estimates what it is meant to estimate.

The correct procedure is to fit the scaler on the training split alone and then apply those stored statistics, unchanged, to the validation and test data:

$$\mu, \sigma \text{ from train} \longrightarrow z_{\text{test}} = \frac{x_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

This mirrors deployment, where future observations must be transformed using statistics fixed at training time, because the future data is not available when the scaler is fitted.

(A) — false, and the standard rationalisation. Determinism is not the issue, and neither is the absence of labels. μ and σ are quantities *derived from data you are pretending not to have seen*, and leakage is about information flow, not about whether a computation involves the target.

(C) — a true observation, but not the objection. It is correct that the training features will no longer have exactly zero mean, since they were centred using a mean computed over a larger set. But this is a harmless numerical side effect of no practical consequence. Mistaking it for the problem is to notice the symptom and miss the disease.

(D) — false. The coefficients remain perfectly interpretable, just expressed relative to statistics that were computed improperly. Interpretability is not what leakage damages; the honesty of the error estimate is. The version of this error most often committed in practice is subtler than the one described: scaling the whole training set once and then running k -fold cross-validation on it. Each fold's held-out portion contributed to the statistics used to transform its own training portion, so the CV estimate is quietly inflated. The scaler belongs *inside* the fold, like every other fitted step — which is precisely what pipeline abstractions in libraries such as scikit-learn exist to enforce.

Q11. You fit a min-max scaler on the training set. A test observation exceeds the training maximum. What happens? C The scaler stores the training minimum and maximum and applies them unchanged. With a training range of $[10, 50]$, a test value of 60 maps to

$$\frac{60 - 10}{50 - 10} = \frac{50}{40} = 1.25$$

The value lands above 1, correctly signalling extrapolation beyond anything seen during training.

This is not a bug. The $[0, 1]$ guarantee was always a statement about the *training* data, and a value outside that interval carries genuine information: the model is being asked to predict in a region it was never fitted on. Silently clipping to 1.0 would discard that signal and, worse, would map every out-of-range value to the same point — so a test value of 60 and one of 6000 would become indistinguishable.

(A) — false as a description of the default, though clipping is available as an option in some implementations. It should be a deliberate choice made when the bounded range is a hard requirement downstream,

not an automatic behaviour, and it comes at the cost of the information just described.

(B) — false, and it is leakage. Refitting on test data is exactly the error of the previous question. It would also mean the transformation applied to the training data no longer matches the one applied at test time, so the model would be evaluated on inputs measured on a different scale from the ones it learned.

(D) — false. The arithmetic is well defined for any input; the transformation simply returns a value outside $[0, 1]$. This behaviour makes min-max scaling a free extrapolation detector: monitoring how often scaled test values fall outside $[0, 1]$, and by how far, is a cheap check for distribution shift in production. Standardization gives the same signal in a different currency — a z -score of 8 on live data is a warning that the incoming distribution has drifted from the one the model was trained on.


VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION
VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION